FH University of Applied Sciences TECHNIKUM WIEN

CNN-BiLSTM for Heart Rate Prediction

Biosignal Acquisition and Analysis

Author: Julian Wollner, Patricia Glavas, Thomas Schwab, Bedran Aksu

Affiliation: FH Technikum Wien

Date: 11.11.2026

1 Introduction

Photoplethysmography (PPG) measures the change of blood volume in the tissue optically. A light source illuminates the skin and a photodiode records the transmitted or reflected light, which is modulated by every heartbeat [1]. Because the sensor is cheap and small, PPG is built into watches, rings and earbuds. Its main weakness is noise: motion, changing contact pressure and low perfusion distort or hide the pulse wave, and the reference measurement for heart rate (HR) remains the electrocardiogram (ECG).

Classic PPG heart-rate estimation finds the dominant frequency in a short window or detects individual pulses, often combined with accelerometer-based motion cancellation such as TROIKA [2]. Deep learning has replaced these hand-made pipelines in recent work. Convolutional neural networks (CNN) learn the shape of the pulse wave directly from the signal or its spectrum (DeepPPG [3]), and CNN-LSTM hybrids add a recurrent layer that models the rhythm across several beats (CorNET [4]). A long short-term memory (LSTM) network [5] keeps a state over the sequence. A bidirectional LSTM (BiLSTM) [6] reads the sequence forwards and backwards, so each time step sees the beats before and after it. BeliefPPG [7] predicts a probability distribution over HR per window and smooths it over time, which also yields an uncertainty estimate.

This report describes a CNN-BiLSTM that estimates HR from a single PPG channel, without accelerometer data. The model predicts a distribution over HR. An ensemble, a quality gate and a Kalman smoother turn the per-window distributions into a continuous HR trace at 2 Hz.

2 Data

Two sub-datasets with simultaneous ECG and PPG are used, both sampled at 100 Hz and stored as EDF files without annotations:

- **Multisite-PPG**: 20 subjects, 519 free-living recordings from four wearables (earring, necklace, ring, watch). Recording length median 4.5 h, partly over 23 h.
- **Wrist-finger-PPG**: 27 subjects with 6 short controlled recordings each (165 files), median 4.0 min (1.8–7.0 min).

A third sub-dataset (wrist PPG during exercise, 8 subjects) was excluded, because only 38 % of its windows had a usable ECG label after quality control (Section 3.1).

After labelling and windowing (Section 3.2), 51 864 windows remain: 37 668 multisite and 14 196 wrist-finger. The label distribution is centred at 60–90 BPM (72 % of all windows); only 2.8 % of the windows are at 110 BPM or more, almost all of them from multisite. The mean label is 83.3 ± 14.6 BPM (multisite) and 79.2 ± 10.4 BPM (wrist-finger).

3 Methods

Figure 1 gives an overview. The ECG is used only to produce the label; the network sees only the PPG.

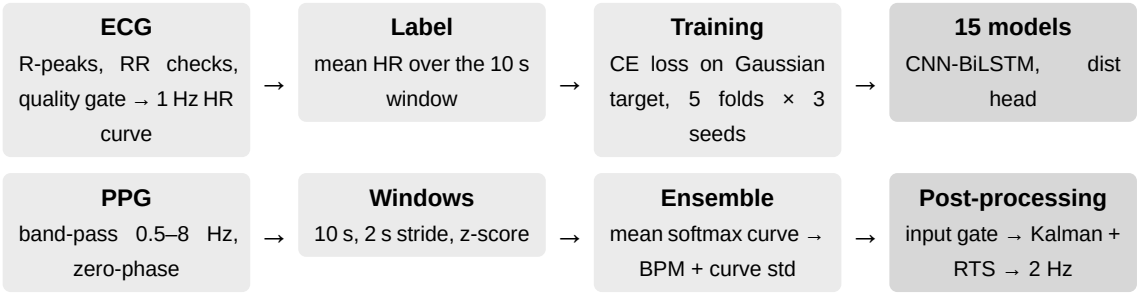


Figure 1: Overview. Top row: label path (training only). Bottom row: input and inference path.

3.1 From ECG to label

R-peak detection. R-peaks are detected in the raw ECG with the Pan-Tompkins algorithm [8] as implemented in NeuroKit2 [9]. For each pair of consecutive peaks, the instantaneous HR is $60/\ \mathrm{RR}$ (RR in seconds), assigned to the time of the second peak. These values are linearly interpolated onto a 1 Hz grid; seconds before the first or after the last anchor are undefined.

Beat checks. A beat is rejected if its instantaneous HR lies outside 30–220 BPM, or if its RR interval differs from the previous one by more than a factor of 1.2 (ratio > 1.2 or $< 1/1.2$). All integer seconds between the two peaks of a rejected beat (inclusive) are set to undefined. No value is interpolated across a rejected beat.

ECG quality gate. Pan-Tompkins produces a regular train of false beats on lead-off ramps that passes the beat checks. Two further rules catch this:

- *Beat agreement (bSQI)* [10]: a second detector (NeuroKit2 default method on the NeuroKit2-cleaned ECG) runs on the same recording. In each non-overlapping 10 s block, beats of the two detectors are matched within 150 ms. If $\mathrm{matched}\ /(n_1 + n_2 - \mathrm{matched}) < 0.8$, or either detector finds fewer than two beats, the whole block is undefined.
- *Flat ECG*: a second is undefined if the peak-to-peak amplitude of the cleaned ECG in it is at most 10 % of the recording median.

Finally, every second in which the raw PPG is constant (sensor off, padding) is undefined, even if the ECG is valid.

Label. The label of a window is the mean of the ten 1 Hz HR values it covers. A window containing any undefined second is discarded.

3.2 From PPG to model input

Filtering. The full PPG recording is band-pass filtered with a 4th-order Butterworth filter, 0.5–8 Hz, applied forwards and backwards (zero phase). The band covers HR from 30 BPM up to the higher harmonics of the pulse wave.

Windowing. Windows of 10 s (1000 samples) start every 2 s. Only windows with a valid label (Section 3.1) are kept for training.

Per-subject cap. Multisite recordings are long, so a few subjects would dominate the training data. For each multisite subject, at most 2000 windows are kept, drawn uniformly

at random without replacement (seed 42), first per file and then per subject. Wrist-finger subjects are not capped (median 541 windows).

Normalisation. Each window is z-scored individually (mean 0, standard deviation 1). Windows with zero standard deviation are left unchanged. This removes the amplitude, which depends on sensor, site and contact, and keeps only the shape.

3.3 Augmentation and balancing (training only)

Gaussian noise. Every time a training window is drawn, noise with $\sigma = 0.05$ (in z-scored units) is added. Validation and test windows are never augmented.

Balanced sampling. The training set holds about three times more multisite than wrist-finger windows. Batches are drawn with a weighted random sampler with replacement, in which each sub-dataset has the same total weight, so each fills about half of every epoch. One epoch draws as many windows as there are real training windows.

Time-warp bootstrap. Low and high HR are rare (Figure 6), and a model trained on the raw distribution pulls its predictions towards the middle. Synthetic windows are therefore created by time-warping real PPG [11] (Figure 2). A source span of $10 \cdot f$ seconds is linearly resampled into one 10 s window. For $f > 1$ the pulse is compressed and the HR rises, for $f < 1$ it falls. The label is f times the HR curve sampled at the ten source times $t_0 + fk$, $k = 0, ..., 9$, averaged. The span is extended by $3f$ seconds on each side, filtered again with the same band-pass after warping, and cropped. The second filter removes energy that warping moves out of the 0.5–8 Hz band and that would otherwise mark the window as synthetic.

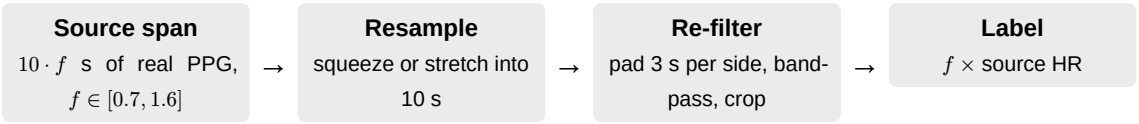


Figure 2: Time-warp bootstrap for synthetic low- and high-HR windows.

Synthetic windows are generated from the training subjects of each fold only. Target bins are 40–50 and 50–60 BPM (both sub-datasets) and 10 BPM bins from 110 to 190 BPM (multisite) or 110 to 130 BPM (wrist-finger). Each bin is filled up to 500 windows per sub-dataset, counting the real training windows already in it. Candidate source starts are integer seconds with a valid 10 s mean HR h. For a bin $[l, u)$, the factor is drawn uniformly from $[\max(l/h, 0.7), \min(u/h, 1.6)]$, and the window is kept only if its label falls into the bin and no second of the source span is undefined. Each source start is used at most five times; a bin that runs out of sources stays short. For the high bins, a source is used only if the dominant frequency of its PPG (Hann-windowed FFT, 0.6–3.5 Hz) lies within 10 % of its ECG HR. Otherwise motion-corrupted PPG would receive a high label without a matching pulse. In the sampler, synthetic windows receive at most 20 % of their sub-dataset's weight.

3.4 Model architecture

The network maps one z-scored 10 s window (1×1000) to 191 logits, one per HR bin of 1 BPM from 30 to 220 BPM. Table 1 lists the layers; the model has 161 791 parameters.

Block	Operation	Output	Params
Input	10 s band-passed PPG at 100 Hz, z-scored	1×1000	–
Conv 1	Conv1d 1 → 32, kernel 5, same padding · BatchNorm · ReLU · MaxPool 2	32×500	256
Conv 2	Conv1d 32 → 64, kernel 5 · BatchNorm · ReLU · MaxPool 2	64×250	10 432
Conv 3	Conv1d 64 → 128, kernel 5 · BatchNorm · ReLU · MaxPool 2	128×125	41 344
BiLSTM	1 layer, 64 hidden units per direction	125×128	99 328
Pooling	mean over the 125 time steps	128	–
Dense	Linear 128 → 32 · ReLU · Dropout 0.3	32	4 128
Head	Linear 32 → 191 (logits)	191	6 303

Table 1: CNN-BiLSTM layers.

CNN encoder. Three convolution blocks with batch normalisation [12] extract the local pulse shape (upstroke, peak, dicrotic notch) and suppress noise. Each max-pooling halves the time axis, from 1000 to 125 steps (80 ms apart); one step sees 36 input samples (0.36 s).

BiLSTM. The sequence of 125 feature vectors is read by a bidirectional LSTM [5], [6]. It models the rhythm across beats, so a corrupted beat can be inferred from its neighbours.

Pooling and head. The BiLSTM outputs are averaged over time, which matches the label (mean HR over the window). A dense layer with dropout [13] and a linear layer produce the 191 logits.

Distribution head. The task is treated as label distribution learning [14]. The softmax of the logits gives a probability curve $p(c)$ over the bin centres $c = 30, ..., 220$. The training target for a label y is a discretised Gaussian

$$q(c) = \exp\!\left(-(c-y)^2/(2\sigma^2)\right) / \sum_{c'} \exp\!\left(-(c'-y)^2/(2\sigma^2)\right), \quad \sigma = 2 \text{ BPM}.$$

Neighbouring bins share the credit, so a prediction 1 BPM off is penalised less than one 30 BPM off. To decode one value, the bins within ±10 BPM of the curve's maximum are kept and their expected value is taken:

$$\hat{y} = \left(\sum_{|c-c^*| \le 10} p(c)c \right) \Big/ \left(\sum_{|c-c^*| \le 10} p(c) \right), \quad c^* = \arg\max_c p(c).$$

A plain mean over all bins would be pulled towards a second mode (harmonic or motion peak), and the argmax alone is quantised to 1 BPM. The standard deviation of the whole curve, $s = \sqrt{\sum_c p(c)(c-\mu)^2}$ with $\mu = \sum_c p(c)c$, serves as the uncertainty score of the window.

3.5 Training

Loss. Cross-entropy between target and prediction per window, $\mathcal{L} = -\sum_c q(c) \log p(c)$, averaged over the batch.

Optimisation. Adam [15], learning rate 10^{-3}, batch size 64, at most 100 epochs. The learning rate is halved when the validation metric has not improved for 5 epochs (minimum 10^{-5}).

Validation and early stopping. After each epoch, the model predicts all validation windows (capped, not augmented). The validation metric is the balanced MAE: the window-pooled MAE of each sub-dataset, averaged over the two sub-datasets. Training stops after 20 epochs without improvement; the checkpoint with the lowest balanced validation MAE is kept, not the last one.

Seeds. Each fold is trained with three seeds (42, 1, 2), which set the weight initialisation, the sampling order and the noise. cuDNN runs in deterministic mode. Training ran on one NVIDIA RTX 3080 GPU.

3.6 Cross-validation

All splits are subject-wise: no subject's windows appear in more than one of train, validation and test. Within each sub-dataset, subjects are assigned to five groups, stratified by the share of their windows at high HR (≥ 130 BPM for multisite, ≥ 100 BPM for wrist-finger). Subjects are sorted by this share, cut into strata of five, shuffled within each stratum (seed 42) and dealt to groups 0–4. This puts the subjects with the most high-HR windows into different groups. Multisite has 4 subjects per group, wrist-finger 6/6/5/5/5.

In fold k, group k is the test set, group $(k+1) \bmod 5$ the validation set and the remaining three groups the training set. Every subject is thus tested once and validated once. Everything that is fitted (bootstrap quotas, gate limit, smoother parameters) uses only the training or validation subjects of the fold. Five folds times three seeds give 15 trained models.

3.7 Inference and post-processing

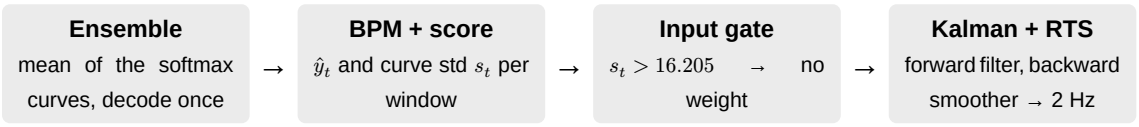


Figure 3: Post-processing of the window predictions of one recording.

Input. A raw PPG signal of any sampling rate is resampled to 100 Hz (polyphase), missing samples are linearly interpolated, and the signal is band-pass filtered and cut into 10 s windows every 2 s as in Section 3.2.

Ensemble. All 15 models predict every window; their softmax curves are averaged [16], and the mean curve is decoded once into $\hat{y}_t$ and its standard deviation s_t.

Input gate. A window with $s_t > 16.205$ BPM is treated as unreliable. The limit comes from cross-validation: for each fold, the limit is the 90 % quantile of s_t over that fold's validation windows (3-seed ensemble); the shipped limit is the median of the five fold limits.

Kalman filter and RTS smoother. The HR over the window sequence of one recording is modelled as a random walk [17]:

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad Q = \left(m \cdot \sigma_{\text{step}}\right)^2,$$

$$\hat{y}_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R_t), \quad R_t = (c \cdot s_t)^2.$$

The step deviation $\sigma_{\text{step}} = 1.0$ BPM per 2 s step is estimated robustly from the training labels (10 s mean labels at 2 s stride, $1.4826 \cdot \text{MAD}$ of the first differences). Gated windows get $R_t = \infty$, so the filter bridges them with the prediction alone. The filter starts at the first ungated measurement with variance 10^6. A forward Kalman pass is followed by a backward Rauch-Tung-Striebel (RTS) pass [18], so each estimate uses past and future windows. The factors $m \in \{1, 2, 4\}$ and $c \in \{0.5, 1, 2, 4\}$ were tuned on the validation subjects (lowest subject-macro balanced MAE); the shipped values are $m = 1$, $c = 0.5$.

Output. The smoothed values sit at the window centres (start + 5 s). They are linearly interpolated to a 2 Hz output grid; before the first and after the last centre the nearest value is held. No output gate is applied, because the interface requires a value every 0.5 s.

3.8 Evaluation

The shipped 15-model ensemble has seen every subject during training and cannot be tested fairly. The performance of the recipe is therefore estimated out-of-fold: each subject is predicted by the 3-seed ensemble of the fold in which it is a test subject, with the gate limit and smoother parameters of that fold. The pipeline runs on every 2 s window within ±5 min of a labelled window, without the per-subject cap, and is scored on all 427 994 labelled windows.

The primary metric is the subject-macro MAE: the MAE per subject, averaged over the subjects of a sub-dataset, then averaged over the two sub-datasets. A long recording thus counts no more than a short one. Further metrics are the window-pooled MAE, the RMSE, the bias (mean of prediction minus truth), Pearson's r, the share of windows within ±5 and ±10 BPM, and Bland-Altman limits of agreement [19].

4 Results

Table 2 compares the raw ensemble with the shipped pipeline. Over both sub-datasets, the subject-macro MAE drops from 4.11 to 3.64 BPM through gate and smoother. On wrist-finger the model is almost exact (1.47 BPM, $r = 0.98$); on multisite it reaches 5.82 BPM ($r = 0.72$). Gating and smoothing help mainly on multisite (6.73 → 5.82 BPM, r 0.57 → 0.72).

Pipeline	Dataset	MAE_{subj}	MAE_{win}	RMSE	Bias	r	±5 / ±10
Raw ensemble	Multisite	6.73	7.49	13.35	+1.81	0.57	62 / 74 %
	Wrist-finger	1.48	1.50	2.53	+0.16	0.97	95 / 99 %
	Both	4.11	4.49	7.94	+0.99	0.77	78 / 87 %
Shipped	Multisite	5.82	6.47	10.84	+1.22	0.72	63 / 77 %
	Wrist-finger	1.47	1.48	2.24	+0.12	0.98	96 / 99.5 %
	Both	**3.64**	3.97	6.54	+0.67	0.85	80 / 89 %

Table 2: Out-of-fold test results (BPM) on 427 994 labelled windows of 47 subjects. Shipped = ensemble + input gate + Kalman/RTS. MAE_{subj}: subject-macro MAE; "Both" averages the two sub-datasets for MAE_{subj}, other columns pool all windows.

The input gate removes 1.5 % of the multisite windows and none of the wrist-finger windows. Scoring only windows whose gated gaps are at most 30 s (98.5 % of the windows) gives 3.56 BPM.

Figure 4 shows predictions against the ECG label. On wrist-finger the points follow the diagonal (slope 0.93). On multisite the fit has a slope of 0.49: a large share of windows is predicted near 80–90 BPM regardless of the true HR. The Bland-Altman analysis (Figure 5) confirms that the bias is small (+1.2 BPM multisite, +0.1 BPM wrist-finger), but the multisite limits of agreement are wide (−19.9 to +22.3 BPM, wrist-finger −4.3 to +4.5 BPM).

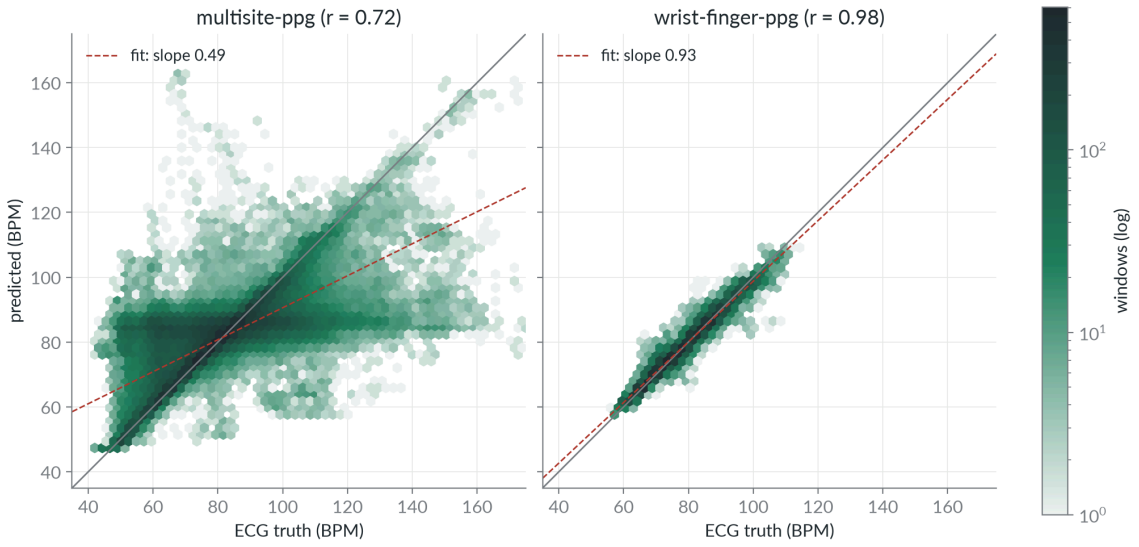


Figure 4: Predicted HR against ECG label, shipped pipeline, all test windows (log density). Grey: identity; red dashed: least-squares fit.

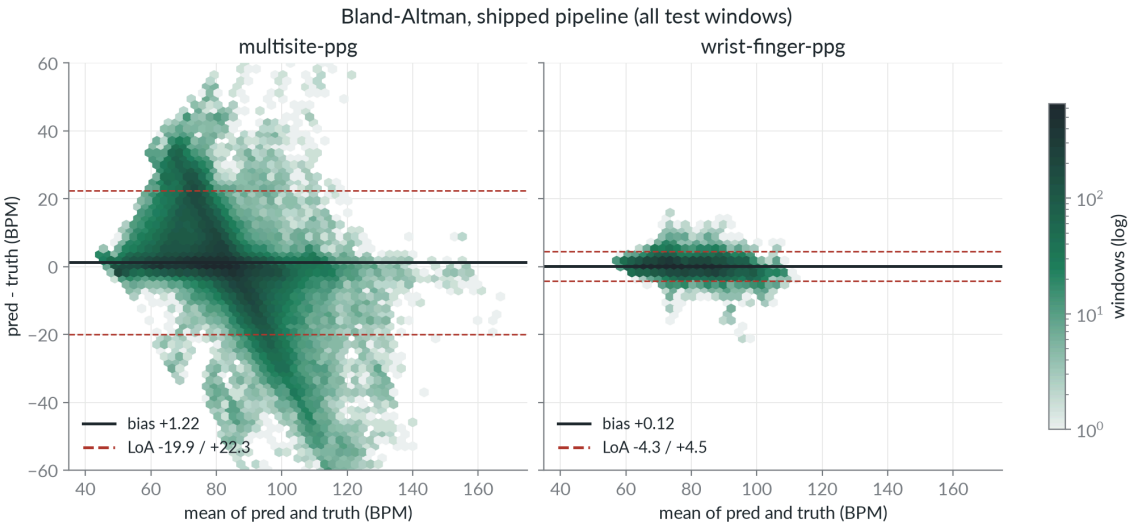


Figure 5: Bland-Altman plots, shipped pipeline. Solid: bias; dashed: 95 % limits of agreement.

Figure 6 breaks the error down by true HR. Between 70 and 90 BPM, where most data lies, the MAE is 2.2–3.7 BPM. Towards the tails the error grows and becomes a bias: 40–60 BPM is over-predicted by 11.4 BPM, and HR above 130 BPM is under-predicted by 42.4 BPM. Smoothing lowers the MAE in every bin by about 1 BPM but does not remove this bias.



Figure 6: Error per HR bin, test windows. Top: MAE of raw ensemble and shipped pipeline (number of windows above the bars). Bottom: bias of the shipped pipeline.

Per subject (Figure 7), every wrist-finger subject stays below 3.4 BPM (median 1.14 BPM). Multisite subjects spread from about 2 to 12.4 BPM (median 5.30 BPM); the error is spread over many subjects rather than caused by one outlier. Across the five fold

ensembles, the subject-macro MAE is 3.39 ± 0.27 BPM on training, 4.00 ± 0.37 BPM on validation and 4.18 ± 0.27 BPM on test subjects (raw ensembles, mean ± sd over folds).

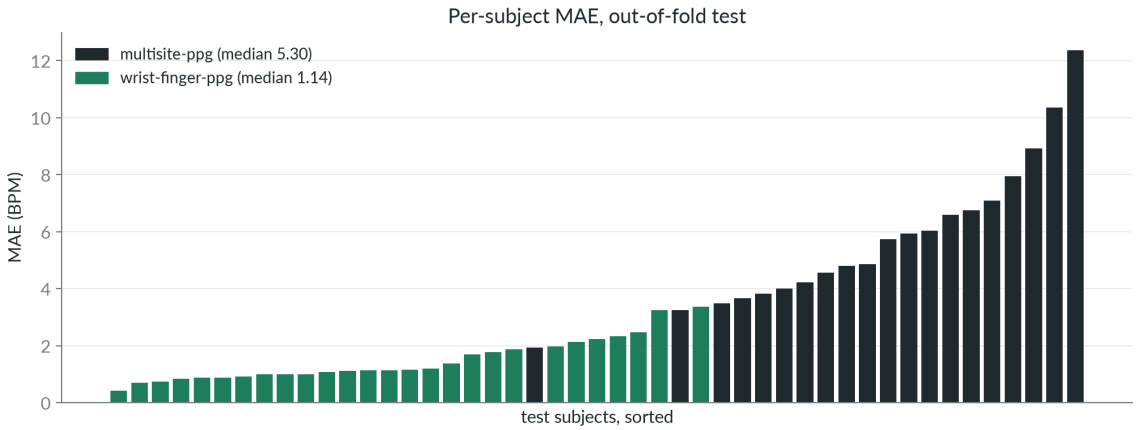


Figure 7: Per-subject MAE, out-of-fold test, shipped pipeline, sorted.

5 Discussion and Conclusion

The CNN-BiLSTM estimates HR from controlled finger and wrist recordings with an error close to the ECG reference (1.47 BPM, 96 % of windows within 5 BPM). Free-living multisite PPG is much harder (5.82 BPM). Even on its own training subjects, the multisite error stays near 6 BPM (5.7 BPM on train vs. 6.9 BPM on test), so the gap is mainly due to signal quality, not overfitting. The authors of the multisite dataset report 2.3–8.7 BPM on the same recordings depending on the body site; their protocol differs, so the numbers are only roughly comparable.

The main failure mode is the pull towards the typical HR. When the PPG shows no clear pulse, the model falls back on the most common labels (80–90 BPM), which produces the horizontal band in Figure 4 and the strong bias at low and high HR in Figure 6. The time-warp bootstrap and the distribution head reduce this effect but do not remove it. Warping also scales systole and diastole equally, whereas a real HR increase mainly shortens diastole, so the synthetic high-HR windows are not fully realistic.

Gate and Kalman smoother lower the MAE by 0.5 BPM without retraining and give each estimate an uncertainty. Since the interface requires a value every 0.5 s, unreliable stretches are bridged by the smoother rather than flagged; a user of the output cannot tell these values apart.

Limitations: no accelerometer input, so motion cannot be modelled directly; labels depend on ECG R-peak detection, and the quality rules discard data rather than repairing it; only 47 subjects from two sources; the shipped ensemble is not refitted on all subjects, and its performance is estimated from the 3-seed fold ensembles. Future work should add motion signals, more high-HR data from real activity, and a time-domain output that can mark uncertain stretches.

Bibliography

[1] J. Allen, “Photoplethysmography and its application in clinical physiological measurement,” *Physiological Measurement*, vol. 28, no. 3, pp. R1–R39, 2007, doi: 10.1088/0967-3334/28/3/R01.

[2] Z. Zhang, Z. Pi, and B. Liu, “TROIKA: A General Framework for Heart Rate Monitoring Using Wrist-Type Photoplethysmographic Signals During Intensive Physical Exercise,” *IEEE Transactions on Biomedical Engineering*, vol. 62, no. 2, pp. 522–531, 2015, doi: 10.1109/TBME.2014.2359372.

[3] A. Reiss, I. Indlekofer, P. Schmidt, and K. Van Laerhoven, “Deep PPG: Large-Scale Heart Rate Estimation with Convolutional Neural Networks,” *Sensors*, vol. 19, no. 14, p. 3079, 2019, doi: 10.3390/s19143079.

[4] D. Biswas *et al.*, “CorNET: Deep Learning Framework for PPG-Based Heart Rate Estimation and Biometric Identification in Ambulant Environment,” *IEEE Transactions on Biomedical Circuits and Systems*, vol. 13, no. 2, pp. 282–291, 2019, doi: 10.1109/TBCAS.2019.2892297.

[5] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[6] M. Schuster and K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997, doi: 10.1109/78.650093.

[7] V. Bieri, P. Streli, B. U. Demirel, and C. Holz, “BeliefPPG: Uncertainty-aware Heart Rate Estimation from PPG Signals via Belief Propagation,” in *Proceedings of the 39th Conference on Uncertainty in Artificial Intelligence (UAI)*, in Proceedings of Machine Learning Research, vol. 216. 2023, pp. 173–183.

[8] J. Pan and W. J. Tompkins, “A Real-Time QRS Detection Algorithm,” *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, 1985, doi: 10.1109/TBME.1985.325532.

[9] D. Makowski *et al.*, “NeuroKit2: A Python toolbox for neurophysiological signal processing,” *Behavior Research Methods*, vol. 53, no. 4, pp. 1689–1696, 2021, doi: 10.3758/s13428-020-01516-y.

[10] Q. Li, R. G. Mark, and G. D. Clifford, “Robust heart rate estimation from multiple asynchronous noisy sources using signal quality indices and a Kalman filter,” *Physiological Measurement*, vol. 29, no. 1, pp. 15–32, 2008, doi: 10.1088/0967-3334/29/1/002.

[11] T. T. Um *et al.*, “Data Augmentation of Wearable Sensor Data for Parkinson's Disease Monitoring using Convolutional Neural Networks,” in *Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI)*, 2017, pp. 216–220. doi: 10.1145/3136755.3136817.

[12] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[14] B.-B. Gao, C. Xing, C.-W. Xie, J. Wu, and X. Geng, “Deep Label Distribution Learning With Label Ambiguity,” *IEEE Transactions on Image Processing*, vol. 26, no. 6, pp. 2825–2838, 2017, doi: 10.1109/TIP.2017.2689998.

[15] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.

[16] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[17] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.

[18] H. E. Rauch, F. Tung, and C. T. Striebel, “Maximum likelihood estimates of linear dynamic systems,” *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965, doi: 10.2514/3.3166.

[19] J. M. Bland and D. G. Altman, “Statistical methods for assessing agreement between two methods of clinical measurement,” *The Lancet*, vol. 327, no. 8476, pp. 307–310, 1986, doi: 10.1016/S0140-6736(86)90837-8.